AI Banking Assistant

Test Execution Guide

```
AI BANKING ASSISTANT — TEST EXECUTION GUIDE

This guide explains how to execute the automated test suite for the AI Banking
Assistant, step by step. The tests cover Use Case 1 (Account Balance Inquiry)
and Use Case 2 (Contract Lookup &amp;amp; Summary).

1. PREREQUISITES

Before running the tests, ensure the following are installed:

  Software Required:
    * Python 3.9 or higher
    * pip (Python package manager)
    * Git (to clone the repository)

  Verify Python installation:
    $ python --version
    Python 3.9.x (or higher)

  Verify pip installation:
    $ pip --version

2. ENVIRONMENT SETUP

Step 2.1: Navigate to the project directory
-------------------------------------------
    $ cd /path/to/Kirodemo

Step 2.2: Create a virtual environment (recommended)
-----------------------------------------------------
    $ python -m venv venv
    $ source venv/bin/activate        # macOS/Linux
    $ venv\Scripts\activate           # Windows

Step 2.3: Install runtime dependencies
--------------------------------------
    $ pip install -r requirements.txt

    This installs:
      - fastapi (web framework)
      - uvicorn (ASGI server)
      - boto3 (AWS SDK)
      - pydantic (data validation)

Step 2.4: Install test dependencies
-----------------------------------
    $ pip install -r requirements-dev.txt

    This installs:
      - pytest (test framework)
      - hypothesis (property-based testing)
      - httpx (HTTP client for FastAPI TestClient)
      - moto (AWS service mocking)

Step 2.5: Verify all packages installed
---------------------------------------
    $ pip list | grep -E "pytest|moto|httpx|fastapi|boto3"

    Expected output (versions may vary):
      boto3            1.x.x
      fastapi          0.x.x
      httpx            0.x.x
```

```
      moto            5.x.x
      pytest          9.x.x

3. TEST FILE OVERVIEW

The test suite is located in the following files:

  File                        Description
  --------------------------  ------------------------------------------------
  test_use_cases.py           Use Case 1 &amp;amp; 2 automated tests (17 tests)
  test_integration_chat.py    Full integration tests for /chat endpoint
  test_setup_infra.py         Unit tests for infrastructure provisioning
  test_integration_infra.py   Integration tests for AWS resources (requires creds)
  test_dynamodb_client.py     Unit tests for DynamoDB client

Primary test file for Use Cases:  test_use_cases.py

  Test Classes:
    * TestUseCase1_AccountBalanceInquiry   (8 tests)
    * TestUseCase2_ContractLookupAndSummary (9 tests)

4. RUNNING THE TESTS — STEP BY STEP

Step 4.1: Run all Use Case tests
---------------------------------
    $ pytest test_use_cases.py -v

    This runs all 17 tests for Use Case 1 and Use Case 2.
    The -v flag enables verbose output showing each test name and result.

Step 4.2: Run only Use Case 1 (Account Balance Inquiry)
--------------------------------------------------------
    $ pytest test_use_cases.py::TestUseCase1_AccountBalanceInquiry -v

    This runs only the 8 tests for Use Case 1.

Step 4.3: Run only Use Case 2 (Contract Lookup &amp;amp; Summary)
-----------------------------------------------------------
    $ pytest test_use_cases.py::TestUseCase2_ContractLookupAndSummary -v

    This runs only the 9 tests for Use Case 2.

Step 4.4: Run a single specific test
-------------------------------------
    $ pytest test_use_cases.py::TestUseCase1_AccountBalanceInquiry::test_uc1_step1_to_7_acc1001_balance_inquiry -v

Step 4.5: Run all project tests (unit + integration mocked)
------------------------------------------------------------
    $ pytest test_use_cases.py test_integration_chat.py test_setup_infra.py -v

Step 4.6: Run with short traceback on failure
----------------------------------------------
    $ pytest test_use_cases.py -v --tb=short

Step 4.7: Run and stop at first failure
----------------------------------------
    $ pytest test_use_cases.py -v -x

5. EXPECTED TEST OUTPUT

When all tests pass, you should see output similar to:

    ============================= test session starts ==============================
    collected 17 items

    test_use_cases.py::TestUseCase1_AccountBalanceInquiry::test_uc1_step1_to_7_acc1001_balance_inquiry PASSED
    test_use_cases.py::TestUseCase1_AccountBalanceInquiry::test_uc1_acc1002_checking_account PASSED
    test_use_cases.py::TestUseCase1_AccountBalanceInquiry::test_uc1_acc1003_investment_account PASSED
    test_use_cases.py::TestUseCase1_AccountBalanceInquiry::test_uc1_step3_intent_classified_as_balance_inquiry PASSED
    test_use_cases.py::TestUseCase1_AccountBalanceInquiry::test_uc1_alt_no_account_id_prompts_user PASSED
    test_use_cases.py::TestUseCase1_AccountBalanceInquiry::test_uc1_alt_account_not_found_escalates PASSED
    test_use_cases.py::TestUseCase1_AccountBalanceInquiry::test_uc1_alt_dynamodb_unavailable_returns_502 PASSED
```

```
    test_use_cases.py::TestUseCase1_AccountBalanceInquiry::test_uc1_response_schema_conforms PASSED
    test_use_cases.py::TestUseCase2_ContractLookupAndSummary::test_uc2_step1_to_7_contract_lookup_with_summary PASSED
    test_use_cases.py::TestUseCase2_ContractLookupAndSummary::test_uc2_second_contract_c456 PASSED
    test_use_cases.py::TestUseCase2_ContractLookupAndSummary::test_uc2_step2_skips_intent_classification PASSED
    test_use_cases.py::TestUseCase2_ContractLookupAndSummary::test_uc2_step6_summary_truncated_to_1024_chars PASSED
    test_use_cases.py::TestUseCase2_ContractLookupAndSummary::test_uc2_alt_contract_not_found_escalates PASSED
    test_use_cases.py::TestUseCase2_ContractLookupAndSummary::test_uc2_alt_bedrock_fails_returns_escalate PASSED
    test_use_cases.py::TestUseCase2_ContractLookupAndSummary::test_uc2_alt_dynamodb_unavailable_returns_502 PASSED
    test_use_cases.py::TestUseCase2_ContractLookupAndSummary::test_uc2_response_schema_conforms PASSED
    test_use_cases.py::TestUseCase2_ContractLookupAndSummary::test_uc2_response_contains_contract_id_in_message PASSED

    ======================== 17 passed in ~4s ========================
```

6. TEST DESCRIPTIONS — USE CASE 1: ACCOUNT BALANCE INQUIRY

#	Test Name	What It Verifies
1	test_uc1_step1_to_7_acc1001_balance_inquiry	Full flow: ACC-1001 returns $5,250.75 USD savings, status AUTO
2	test_uc1_acc1002_checking_account	ACC-1002 returns $1,200.00 USD checking, status AUTO
3	test_uc1_acc1003_investment_account	ACC-1003 returns $48,000.00 USD investment, status AUTO
4	test_uc1_step3_intent_classified_as_balance_inquiry	Bedrock is called for intent classification
5	test_uc1_alt_no_account_id_prompts_user	No ACC-ID in message prompts user to provide one
6	test_uc1_alt_account_not_found_escalates	Unknown account returns ESCALATE
7	test_uc1_alt_dynamodb_unavailable_returns_502	DynamoDB down returns HTTP 502
8	test_uc1_response_schema_conforms	Response has correct JSON schema

7. TEST DESCRIPTIONS — USE CASE 2: CONTRACT LOOKUP & SUMMARY

#	Test Name	What It Verifies
1	test_uc2_step1_to_7_contract_lookup_with_summary	Full flow: C123 returns with AI-generated summary, status AUTO
2	test_uc2_second_contract_c456	Second contract (C456) works
3	test_uc2_step2_skips_intent_classification	contract_id bypasses the intent classifier entirely
4	test_uc2_step6_summary_truncated_to_1024_chars	AI summary is truncated to max 1024 characters
5	test_uc2_alt_contract_not_found_escalates	Missing contract returns ESCALATE
6	test_uc2_alt_bedrock_fails_returns_escalate	Bedrock error returns ESCALATE
7	test_uc2_alt_dynamodb_unavailable_returns_502	DynamoDB down returns HTTP 502
8	test_uc2_response_schema_conforms	Response has correct JSON schema
9	test_uc2_response_contains_contract_id_in_message	Success message includes the contract ID

8. HOW THE TESTS WORK (TECHNICAL DETAILS)

Mocking Strategy:

  The tests use "moto" to mock AWS DynamoDB locally. No real AWS credentials
  or network access is needed to run the unit/use-case tests.

  - DynamoDB tables are created in-memory using moto's @mock_aws decorator

```
  - Bedrock API calls are mocked using unittest.mock.patch
  - FastAPI is tested using the built-in TestClient (no server needed)

  Test Data:
  ----------
  Accounts Table (mocked):
    ACC-1001: $5,250.75 USD, savings
    ACC-1002: $1,200.00 USD, checking
    ACC-1003: $48,000.00 USD, investment

  Contracts Table (mocked):
    C123: $50,000, 5% interest, 5 years
    C456: $120,000, 3.5% interest, 10 years

  No AWS credentials required:
  ----------------------------
  The tests set mock AWS credentials automatically via a pytest fixture.
  You do NOT need real AWS access to run these tests.

9. TROUBLESHOOTING

Problem: ModuleNotFoundError: No module named 'moto'
  Solution: pip install moto

  Problem: ModuleNotFoundError: No module named 'httpx'
  Solution: pip install httpx

  Problem: ModuleNotFoundError: No module named 'fastapi'
  Solution: pip install -r requirements.txt

  Problem: ImportError from project modules (models, exceptions, etc.)
  Solution: Run pytest from the project root directory:
            cd /path/to/Kirodemo &amp;amp;&amp;amp; pytest test_use_cases.py -v

  Problem: Tests hang or timeout
  Solution: Ensure no real AWS calls are being made. The @mock_aws decorator
            should intercept all boto3 calls. Check that patches are correct.

  Problem: "collected 0 items"
  Solution: Ensure file is named test_use_cases.py (starts with test_)
            Ensure test methods start with test_

10. RUNNING TESTS IN CI/CD

For CI/CD pipelines (GitHub Actions, CodePipeline, etc.), use:

    $ pip install -r requirements.txt
    $ pip install -r requirements-dev.txt
    $ pytest test_use_cases.py -v --junitxml=test-results.xml

  The --junitxml flag generates a JUnit XML report compatible with most
  CI/CD systems for test result visualization.

  Exit codes:
    0 = All tests passed
    1 = Some tests failed
    2 = Test execution error (e.g., import error)

11. QUICK START SUMMARY

# One-time setup:
  pip install -r requirements.txt
  pip install -r requirements-dev.txt

  # Run all Use Case tests:
  pytest test_use_cases.py -v

  # Run just Use Case 1:
  pytest test_use_cases.py::TestUseCase1_AccountBalanceInquiry -v

  # Run just Use Case 2:
  pytest test_use_cases.py::TestUseCase2_ContractLookupAndSummary -v
```

```
  # Expected result: 17 passed

END OF GUIDE
```